Home Exam MVE441

Arvid Kolstad

June 5, 2026

1 Question 1(a)

The dataset used was B11, containing 7680 training points with 820 input features across 7 classes. Label distribution is shown in table 1. Input values range from -12.725 to 14.072, with a mean of 0.419 and standard deviation of 3.328.

For dimensionality reduction, PCA was first applied but showed no clear elbow. Instead a $\frac{1}{x}$-shaped variance curve, indicating high variance across all features without class-specific information. Since PCA is unsupervised, F-statistics (sklearn.feature_selection.r_regression) were used instead to identify class-correlated features. A clear elbow appeared at around 20 features (figure 1), which were adopted as the primary features for the remainder of the project. Since the F-test is linear, mutual information (`sklearn.feature_selection.mutual_info_regression`) was also applied to catch any nonlinear correlations, but it identified the same features. All selected features had p-values well below 0.05. To prevent adding features with 1 to 1 correlation, a correlation matrix were created (table 2), were it's clear that no featuers have a correlation of 1 between each other.

Three models were selected: Logistic Regression (PyTorch), kNN (sklearn), and Regularized Discriminant Analysis (NumPy). Logistic Regression provides a strictly linear decision boundary, RDA models each class with Gaussian distributions and adapts well to varying data sizes, and kNN makes no distributional assumptions while capturing highly complex boundarie, three contrasting approaches.

Model evaluation followed the same nested cross-validation pipeline as project 3: the inner kCV ran 3 times, the outer 5 times, repeated 10 times to assess variance, were the kCV always used stratification to create folds. Hyperparameters were tuned using Bayesian optimisation (scikit-optimize), with 20 evaluations per outer fold were the 10 first had a focus on exploration and 10 last had a focus on performance. Models were evaluated on accuracy and macro F1-score, where each class contributes equally regardless of its size. This was done to have a score that showed if a model was down-prioritizing minority classes. Model comparison used Bayesian analysis (Baycomp), with the ROPE determined by running the pipeline on randomly labelled data and set such that the mean probability of equivalent performance exceeded 95%, and this was set to 4%.

The results of the pipeline can be found in figure 2. We can see that the performance is around high 80% to low 90% for the 200 sample size, and goes rapidly up to around 95% for 1000 sample sizes and from there slowly creeps upward from this. We also see that the variance of the scores are slowly reduces with the increase of sample size which is expected. Both scores is quite similar with the exception that f1-score is a bit lower for Logistic Regression and kNN in the beginning. This makes sense if we look at the class accuracy in figure 3. We can see that for 200 samples both escpecially kNN had a hard time classifying class 3 which makes sense as this is by far the smallest class in the dataset. If we look at the variance of the accuracy of class 3 we can also see that his is the largest for all the models when the sample size is small, which is expected. Looking at figure 4 we can see that in the beginning the Regularized Discriminant

Analysis is the clear winner but around 1000 samples they all score within the rope. This is expected because of RDAs ability to adapt it self after small sample sizes.

1.1 Conclusions

- The dataset is fairly balanced were most of the classes has appears around 15% with the exception of class number 3 that only has 6.2% of the labels.
- The index of features that were relevant for the classification task where the following: 7, 37, 100,141, 158, 179, 210, 228, 276, 308, 410, 427, 519, 550, 558, 597, 752, 762, 796, 809. Checking the correlation matrix for these matrices confirmed that none of these features were exactly the same.
- The models does a good job of classifying the dataset with accuracy and F1 scores around 90% for most of the sample sizes. KNN and Logistic Regression does a bit worse on the smallest sample sizes compared to RDA but this expected because of RDAs ability to adapt with LDA and Native Bayes which is known to performe great on smaller sample sizes.
- For smaller sample sizes the accuracy and performance for class 3 is noticable worse than for other classes which is expected. For larger sample sizes the hardest class instead becomes class 7 for all the models. This could be because of the class distribution in features.

Class	**1**	**2**	**3**	4	**5**	**6**	**7**
Fraction	15.6%	15.6%	6.2%	15.8%	14.1%	15.6%	17.2%

Table 1: This table shows the balance over the dataset. The smallest class of the dataset is class 3 and the largest class is 7.

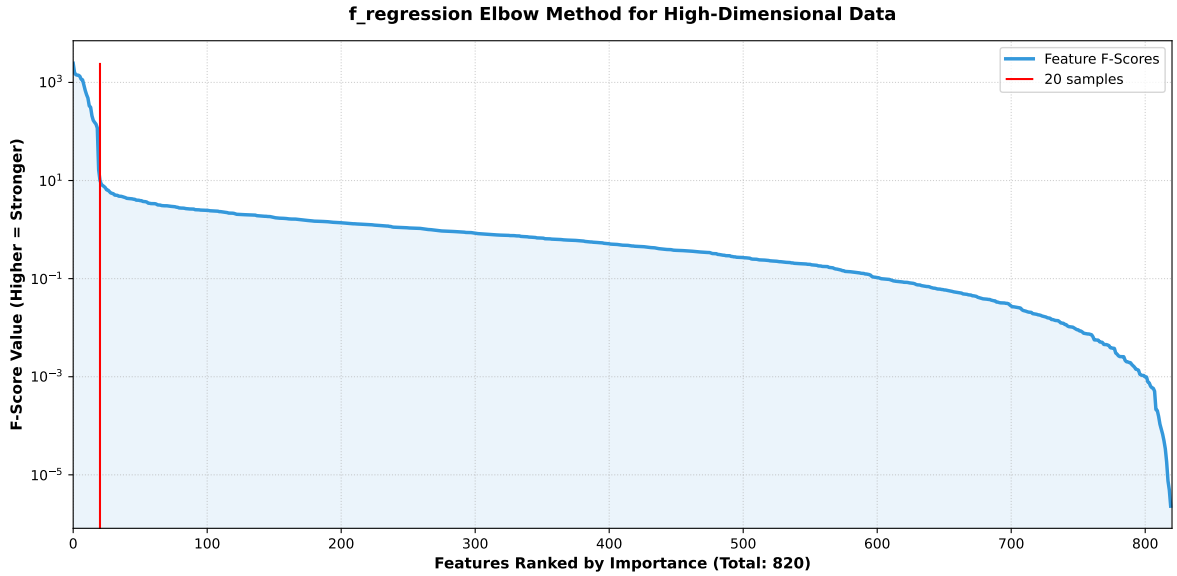

Figure 1: In this figure we can see the F-Score plotted from highest to lowest. We can see a clear elbow after 20 features, and therefore these are the features we will use in the rest of the project.

	7	**37**	**100**	**141**	**158**	**179**	**210**	**228**	**276**	**308**	**410**	**427**	**519**	**550**	**558**	**597**	**752**	**762**	**796**	**809**
7	1.00	0.18	0.49	0.33	-0.40	0.59	-0.39	0.24	-0.27	0.51	-0.19	-0.06	-0.08	0.33	-0.33	-0.35	0.18	0.69	-0.21	-0.05
37	0.18	1.00	0.06	-0.12	0.41	0.19	0.23	0.32	0.54	0.37	-0.50	-0.35	-0.30	-0.19	0.48	-0.18	-0.27	0.18	-0.08	-0.21
100	0.49	0.06	1.00	0.16	-0.37	0.51	-0.52	0.24	-0.15	0.54	-0.39	-0.10	-0.16	0.51	-0.30	-0.47	-0.05	0.60	-0.18	-0.19
141	0.33	-0.12	0.16	1.00	-0.39	0.41	-0.24	-0.01	-0.31	0.31	0.19	0.23	0.24	0.31	-0.28	-0.13	0.28	0.46	0.24	0.03
158	-0.40	0.41	-0.37	-0.39	1.00	-0.30	0.50	0.07	0.72	-0.18	-0.28	-0.31	-0.15	-0.37	0.69	0.17	-0.46	-0.45	0.17	-0.22
179	0.59	0.19	0.51	0.41	-0.30	1.00	-0.40	0.23	-0.07	0.56	-0.37	-0.14	-0.04	0.31	-0.21	-0.50	0.10	0.69	-0.03	-0.20
210	-0.39	0.23	-0.52	-0.24	0.50	-0.40	1.00	0.02	0.36	-0.20	0.16	0.07	-0.06	-0.40	0.51	0.41	-0.22	-0.45	0.01	0.13
228	0.24	0.32	0.24	-0.01	0.07	0.23	0.02	1.00	0.23	0.47	-0.34	-0.22	-0.24	-0.06	0.14	-0.17	-0.18	0.24	-0.14	-0.03
276	-0.27	0.54	-0.15	-0.31	0.72	-0.07	0.36	0.23	1.00	0.12	-0.53	-0.38	-0.27	-0.36	0.69	-0.05	-0.51	-0.21	0.13	-0.31
308	0.51	0.37	0.54	0.31	-0.18	0.56	-0.20	0.47	0.12	1.00	-0.45	-0.16	-0.26	0.19	-0.05	-0.44	-0.10	0.67	-0.18	-0.11
410	-0.19	-0.50	-0.39	0.19	-0.28	-0.37	0.16	-0.34	-0.53	-0.45	1.00	0.52	0.35	-0.01	-0.25	0.52	0.37	-0.32	0.07	0.46
427	-0.06	-0.35	-0.10	0.23	-0.31	-0.14	0.07	-0.22	-0.38	-0.16	0.52	1.00	0.29	0.18	-0.27	0.29	0.30	-0.10	0.17	0.38
519	-0.08	-0.30	-0.16	0.24	-0.15	-0.04	-0.06	-0.24	-0.27	-0.26	0.35	0.29	1.00	0.19	-0.14	0.16	0.22	-0.09	0.26	0.06
550	0.33	-0.19	0.51	0.31	-0.37	0.31	-0.40	-0.06	-0.36	0.19	-0.01	0.18	0.19	1.00	-0.38	-0.23	0.14	0.37	0.08	-0.16
558	-0.33	0.48	-0.30	-0.28	0.69	-0.21	0.51	0.14	0.69	-0.05	-0.25	-0.27	-0.14	-0.38	1.00	0.15	-0.41	-0.34	0.11	-0.16
597	-0.35	-0.18	-0.47	-0.13	0.17	-0.50	0.41	-0.17	-0.05	-0.44	0.52	0.29	0.16	-0.23	0.15	1.00	-0.04	-0.54	0.04	0.34
752	0.18	-0.27	-0.05	0.28	-0.46	0.10	-0.22	-0.18	-0.51	-0.10	0.37	0.30	0.22	0.14	-0.41	-0.04	1.00	0.19	0.02	0.23
762	0.69	0.18	0.60	0.46	-0.45	0.69	-0.45	0.24	-0.21	0.67	-0.32	-0.10	-0.09	0.37	-0.34	-0.54	0.19	1.00	-0.22	-0.14
796	-0.21	-0.08	-0.18	0.24	0.17	-0.03	0.01	-0.14	0.13	-0.18	0.07	0.17	0.26	0.08	0.11	0.04	0.02	-0.22	1.00	-0.10
809	-0.05	-0.21	-0.19	0.03	-0.22	-0.20	0.13	-0.03	-0.31	-0.11	0.46	0.38	0.06	-0.16	-0.16	0.34	0.23	-0.14	-0.10	1.00

Table 2: A table over the correlation matrix for the 20 features that was picked out from the F-Test. No other correlation is close to 1.00 in this table which shows that all of these features are needed for the classificationt task.

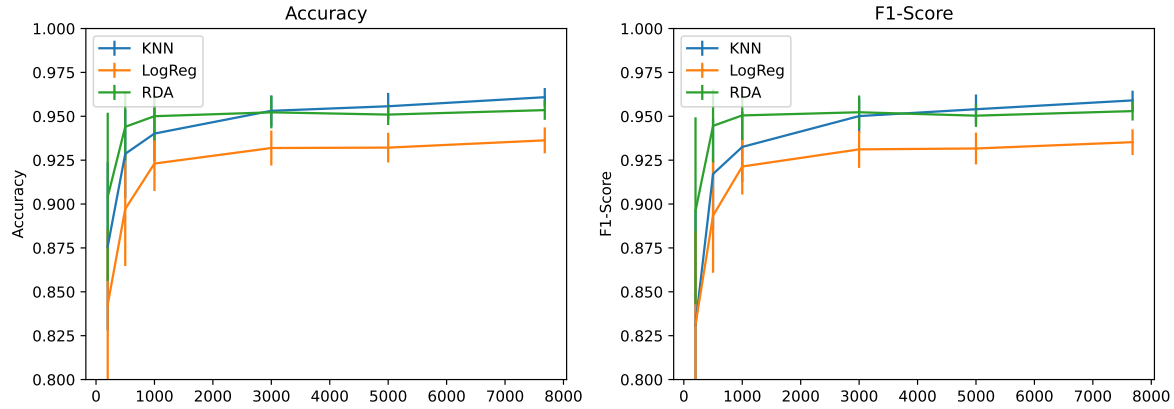

Figure 2: In this figure presents the performance for the diffetent models after running the pipeline over different sample sizes. The left plot displays the accuracy score and the right plot shos the F1-score. We can see that Logistic Regression is performing the worst, and the KNN and RDA is almost similar in their performance over different sample sizes.

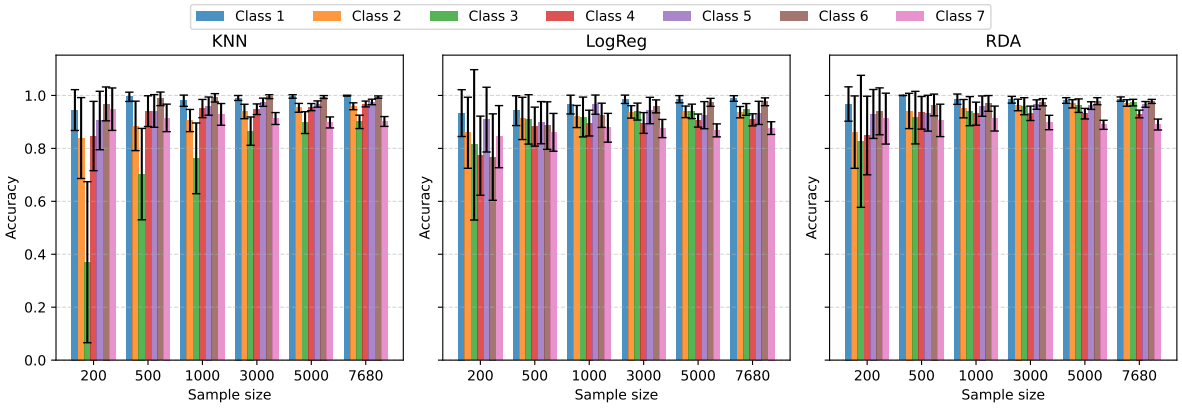

Figure 3: In this figure we can see each models accuuracy per class. While performing the worst for smaller sample sizes, escpecially for class 3, the accuracy is quite similar for higher sample sizes.

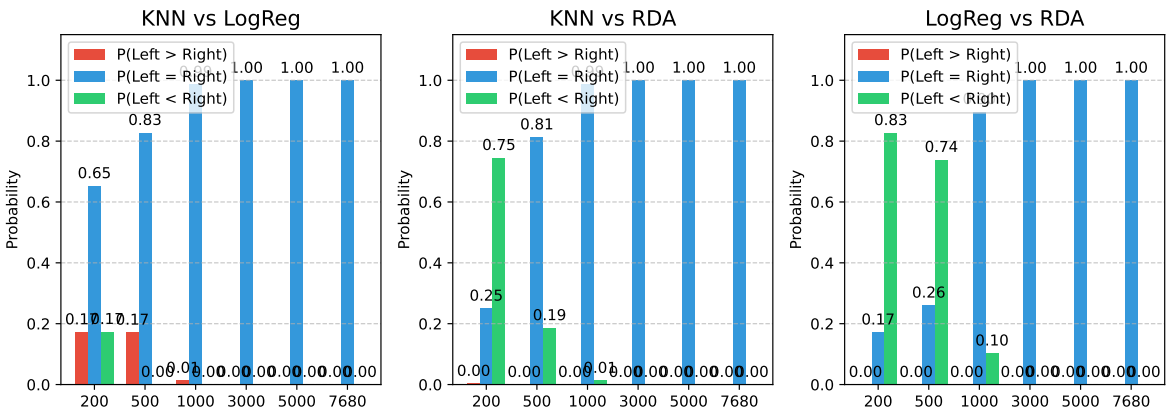

Figure 4: In this picture we can see the bayesian comparisons between the models, were the performance score used is F1-score. While RDA is performing the best in the beginning, the KNN and LogReg is fast to catch up around 1000 in sample size. The rope that was used for the comparison were 4%.

2 Question 1(b)

To address this question, a controlled disruption was introduced to the dataset to intentionally degrade model performance. Specifically, a variable number of randomly selected features from the original 820 features were injected as noise to study the models' robustness. The evaluation pipeline described in Question 1(a) was utilized to measure performance. For each iteration of the pipeline, a new set of random features was sampled to ensure an unbiased and comprehensive assessment of the models' behavior. All 7,600 samples were used across all runs to obtain the most accurate measure of performance degradation. To evaluate the impact, we analyze the performance metrics across all models and perform a Bayesian analysis to compare the F1-scores. The Region of Practical Equivalence (ROPE) for this analysis was defined based on the standard deviation of the original baseline model.

Figure 5 illustrates the performance of the models as a function of the increasing number of noisy features used during training and classification. RDA demonstrates notable robustness against this noise; in fact, for the first few added features, the model's performance increases in terms of both accuracy and F1-score. This behavior is expected given RDA's capacity to adapt to noisy dimensions by tuning its hyperparameters toward LDA or Naive Bayes. This mechanism acts similarly to an ridge regularizer on these parameters, allowing the model to potentially leverage these features rather than allowing them to obscure the class signal. This phenomenon is clearly visible in Figure 6, where the model initially leverages the injected features to its advantage.

As anticipated, the kNN model exhibited the sharpest decline in performance. As shown in Figure 5, the mean values for both accuracy and F1-score drop after the introduction of just a single noisy dimension, and performance continues to monotonically degrade as more dimensions are added. Furthermore, the Bayesian comparison in Figure 6 reveals that the baseline model achieves a higher probability of superiority after only 5 additional dimensions. This confirms that kNN is highly sensitive to noise, likely because the irrelevant dimensions distort the distance metrics, thereby disrupting the local neighborhood structure and degrading classification accuracy.

Logistic Regression also failed to benefit from the additional dimensions. Figure 5 shows that the model's accuracy and F1-score begin to decline after approximately 3 to 5 extra features are introduced. While it proves more resilient than kNN, its performance falls significantly short of RDA. This outcome was somewhat surprising, as Logistic Regression was expected to maintain stability due to its Ridge regularization; however, this penalty proved insufficient to counteract the noise. Figure 6 confirms that the model's performance becomes significantly worse after the introduction of roughly 15 to 20 extra features.

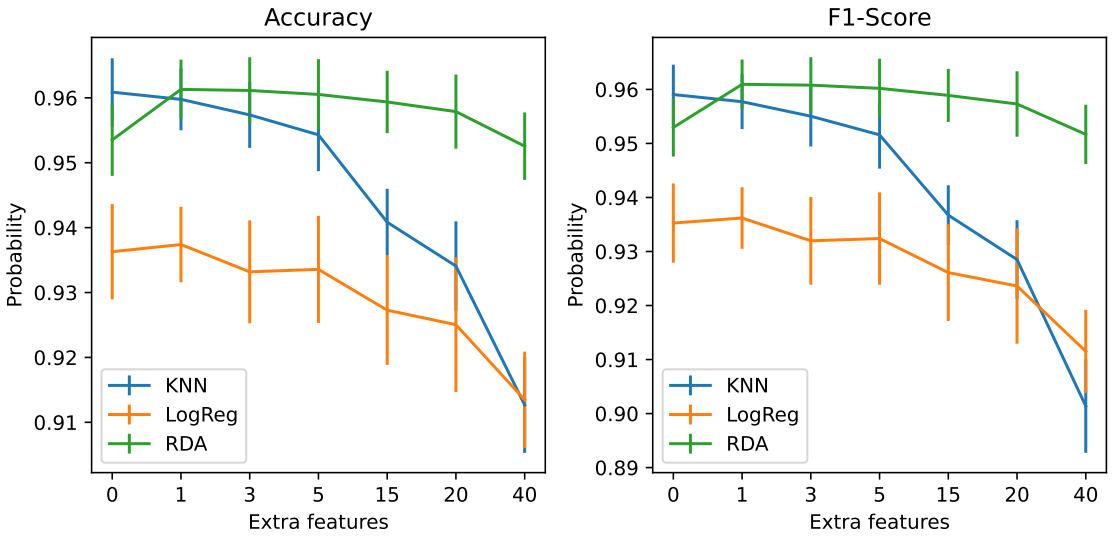

Figure 5: This figure shows the performance over extra features that was added to the inputs. You can see that the RDA model performance was not noticable compared to the other models. The kNN had the worst performance drop out of the models which was expectend because of the models sensitivity to noise.

2.1 Conclusions

- The introduction of random features was sufficient to degrade the performance of several models.
- RDA demonstrated the highest robustness, initially showing an improvement in accuracy before the trend dissipated and performance reverted toward its baseline as more features were added.
- kNN suffered the most severe performance degradation, which aligns with expectations given the model's inherent sensitivity to noise.
- The extent to which Logistic Regression was affected was unexpected. While Ridge regularization was anticipated to mitigate the impact of noise, it proved insufficient, suggesting that Lasso regularization might have been more effective at feature selection in this scenario.

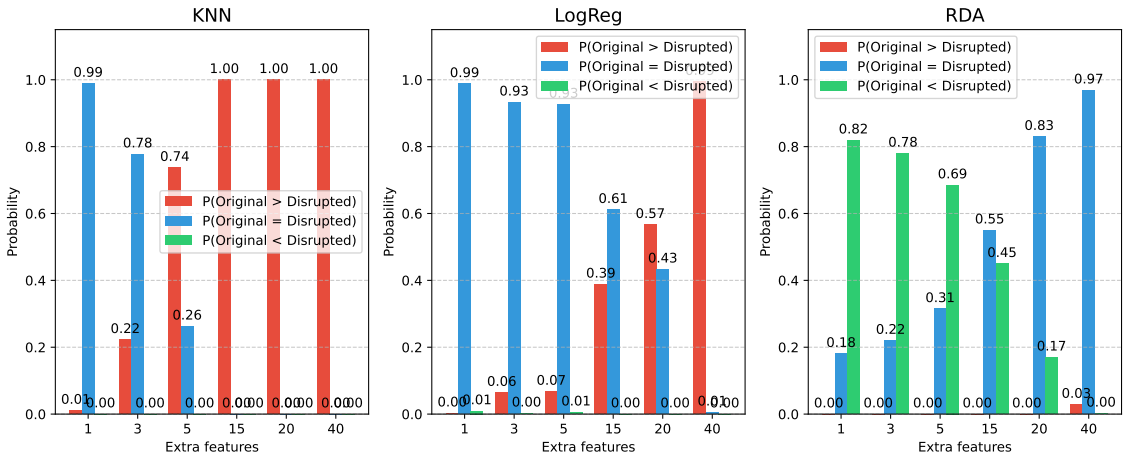

Figure 6: This figure shows the comparison with baysian analysis between the models original performance and the performance with added features. We can see that for the kNN and the LogReg the original model is performing better after a couple of added features, but RDA keeps it performance steady and even improves in the beginning.

3 Question 1(c)

The question here is to conclude if the dataset that is being used in this project contains any mislabels. To do this, a procentage of how sure every model was on its prediction was obtained. From Logistic Regression the softmax from each of it's 7 outputs was taken, for kNN it uses the number of neighbors in that class divided by the total number of neighbors and RDA calculates its confidence by measuring how close a data point is to each class center, adjusted for the regularized shape of the cluster, combining that distance with the overall commonness of the class, and using a softmax function to turn those scores into a final percentage split.

To create a method that can find mislabels in the data, a threshold of 5% was set. If a model clasifies a data point, which does not agree with the data points label, and the probability of the model classifying this as the actuall label is lower than the threshold, then it will mark that point. If all the models mark the data point, then the the label will get classified as a mislabeled point. The false positive rate of this is calculated as following given that the models prediction is correct and that the models are independent of each other:

$$P(\text{model mislabeling a point}) = (1 - (1 - threshold)^{10})^3 \approx 40.13\% \tag{1}$$

which makes all three models have a false positive at the same time as $(40.13\%)^3 = 6.5\%$ which makes it of course possible but not a big part of the data set anyway.

The result of this study can be found in table 3, were 75 mislabels were found and 195 of my 200 labels were correctly identified as wrong. This gives a false negative of about 2.5% which is great. In table 4 you can see the differnt types of mislabelings. The most common mislabeling is the class 4 mislabeled as a 4, and the other mislabels is just one or two.

3.1 Conclusions

- The model that was created was abel, given that the prediction from the model is correct, with a calculated false positive of 6.5% and a experimental value of false negative of 2.5%.

Mislabels Original	**Mislabels Faked**
75	195/200

Table 3: Table over the mislabels found from the original data and the mislabels that the models caught that was introduced by me.

Labeled Class	**Probable Class**	**Number of times**
6	4	57
2	3	4
4	2	3
4	6	2
3	2	2
1	3	2
2	5	2
4	0	1
3	1	1
3	5	1
Total Mislabels		**75**

Table 4: This table summarizes the mislabelings that the models found and what the models thougt was the correct class. It's clear that the class with the most mislabelings were class 6 with a total of 57 mislabeling.

4 Question 1(d)

In this question the two best performing models prediction confidence were to be studied. The first thing to do is to investigate how similar the training data and the test data is. This was done by training a random forest. Random forest is choosen because it's easy to see directly, by looking at the feature importance, what features that differes between the data sets. So what was done was putting 0-labels on the training data and 1-labels on the test data to see the model can classify after training. So by running the model in the pipeline used in Question 1(a) to determine to see the performance of the model. What we will focus on is the ROC-AUC score. This is a score that tells us how well the model understand a binary classifcation task, were a score of 0.5 tells you that the model is just guessing randomly and a score of 1.0 is a perfect model have a 100% accuracy in it's predictions. In the table 5 we see that the mean of the AUC-score is 0.539 with a standard devation of 0.065. This tells us that the model is just guessing and that te test data is very similar to the training data. This means that if our predictions are good on the training data, we should be able to trust these on the test data too.

To meassure how good a model is to know its prediction accuracy we use the Expected Calibration Error which is defined as the following:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right| \tag{2}$$

where N is the total number of samples, B_m represents the set of indices of samples whose prediction confidence falls into the m-th interval, $|B_m|$ is the number of samples in bin B_m, $\mathrm{acc}(B_m)$ is the true accuracy of the predictions in bin B_m, and $\mathrm{conf}(B_m)$ is the average confidence of the predictions in bin B_m. In figure 7 we can see this value ploted for kNN and RDA over different sample sizes. We see that the error is clearly going down with an increasing sample size and both the kNN and the RDA is well below 3% mark which is good. We can also see that in figure 8 how well they score across different confidence ranges. What we can see for 7680 samples is that the RDA is almost perfect in it's guessing and that almost all of it's confidence is in 0.9 to 1.0 range. This is good for this kind of score for the ECE. The kNN is not as good as the kNN but still does not score too bad either. This is probably an effect of the data distribtution is fitting the RDA model very good for this classification. All this means that we should be able to trust our predictions in for the testdata.

In figure 9 we can see that the balances for the different classes are almost the same except the classes 3 and 6. Class 3 is also the minority class in our training data. This is kind almost expected because if we look at figure 10 we see that the kNN is not that confidente with its classifications of class 3. This means that if we can trust the confidence of out models, the more correct class distributions should be the one of RDA because of the uncertanty in the kNNs classifications. Why de classifications confidence is so low for class 3 for the kNN is probably because of it's low concentration in the training data which makes makes it a harder class for kNNs classification strategy.

4.1 Conclusions

- The difference between the training set and the test set is small, because of the inability to classify them.
- The Expected Calibration Error decreases with sample size and for larger sample sizes reaches as low as 2-3% for the kNN and RDA.
- These two findings together makes it so we should be able to trust our classifications for the test sets.
- The models classifies the class balance almost the same with a smaller difference between class 3 and 6. This is explained by class 3 being a minority class and the low confidence in classifications from the kNN.

AUC- mean	AUC - STD
0.539	0.065

Table 5: Table over the AUC score that was obtained after training a random forest to distinguish between the training set and test set. This shows that the model can't distinguish between these sets which means that we can trust out predictions.

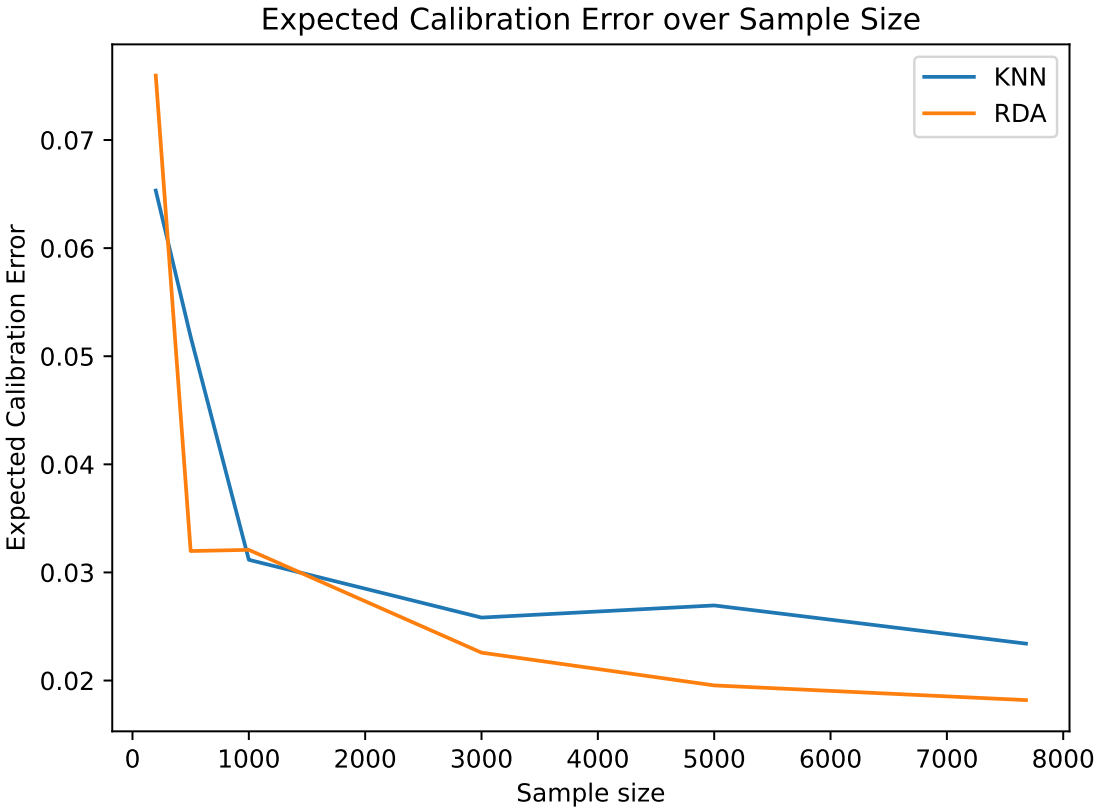

Figure 7: In this plot we can see the Expected Calibration Error ploted over the sample size. We can see that the accuracy increases with sample sizes but that RDA is slightly better when having enough training data.

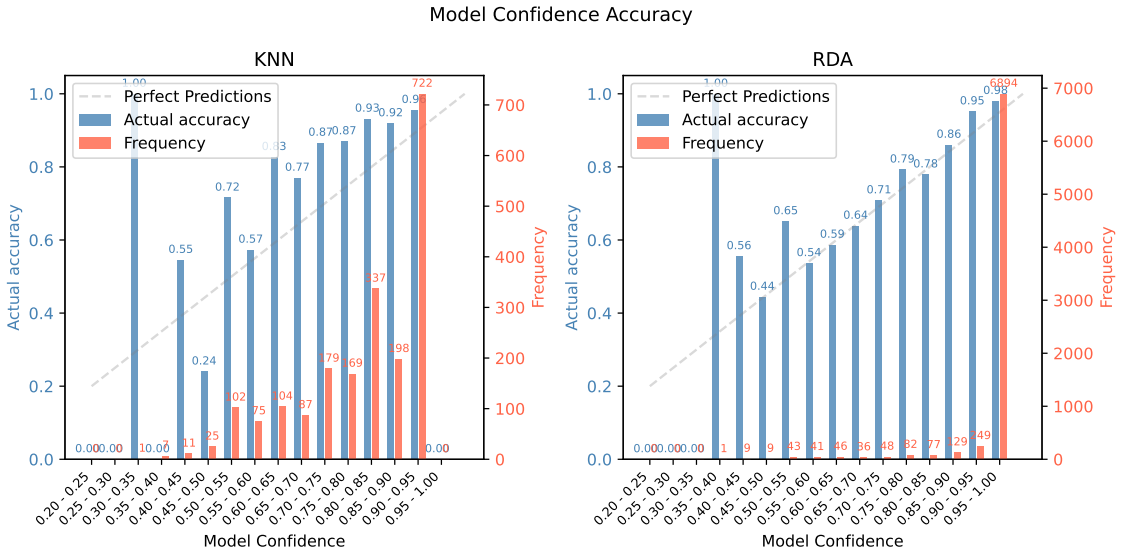

Figure 8: In this figure the blue bars shows the models own confidence in its predictions against the real accuracy of those predictions. We can also see the frequency of the models confidence in the red bars. For kNN we see that the highest frequency is the confidence 0.9-1.0 which is good. But it also has a lot of guesses lower than this. On the other hand RDA is almost always certaion of its classifications and when it isn't it has a good accuracy of it's guesse and has a almost perfect prediction.

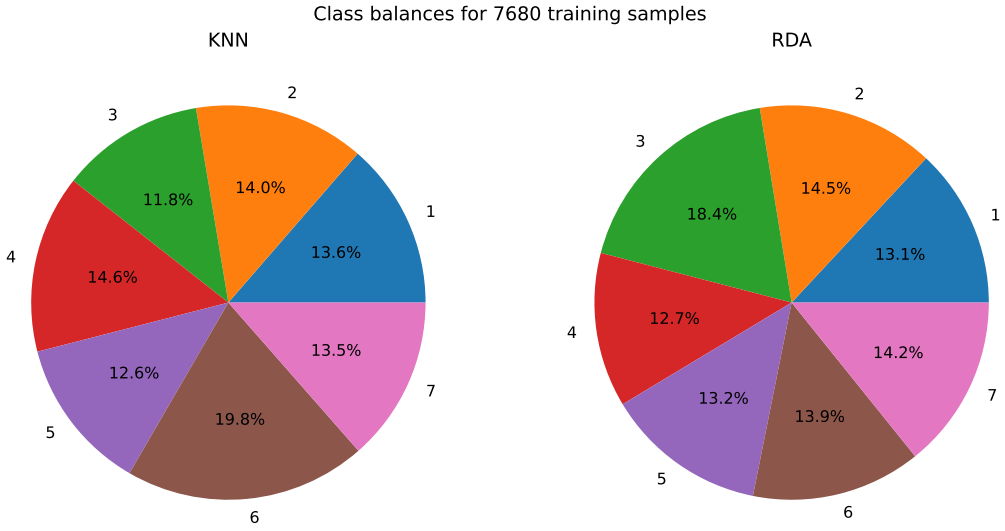

Figure 9: Here are the models classifications balance for the two classes. The biggest differance between the clasifications is class 3 and class 6. Otherwise are the fractions almost the same.

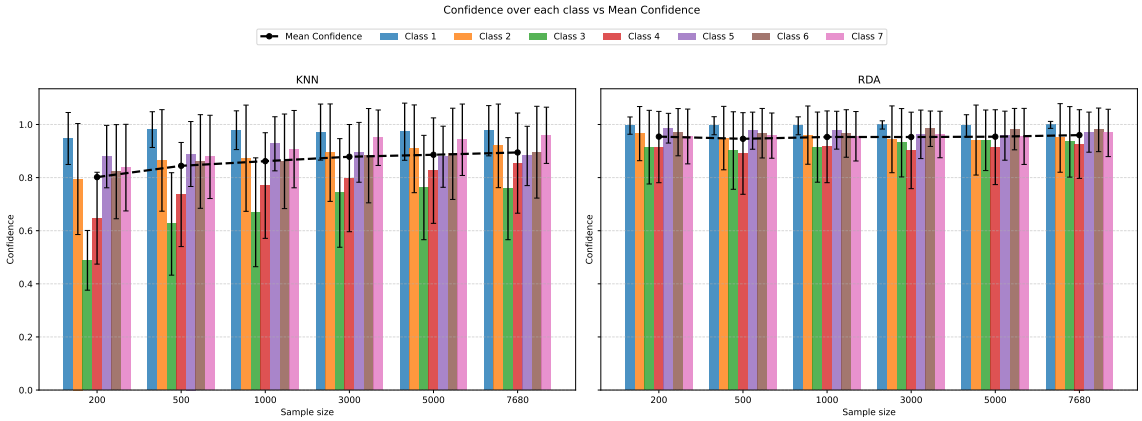

Figure 10: Here we see the classification confidence for each class over different sample sizes of training. We see that class 3 is lowest for the kNN which could explain the difference that was seen in the class balance predictions.

5 Question 2(a) and 2(b)

5.1 Conclusions

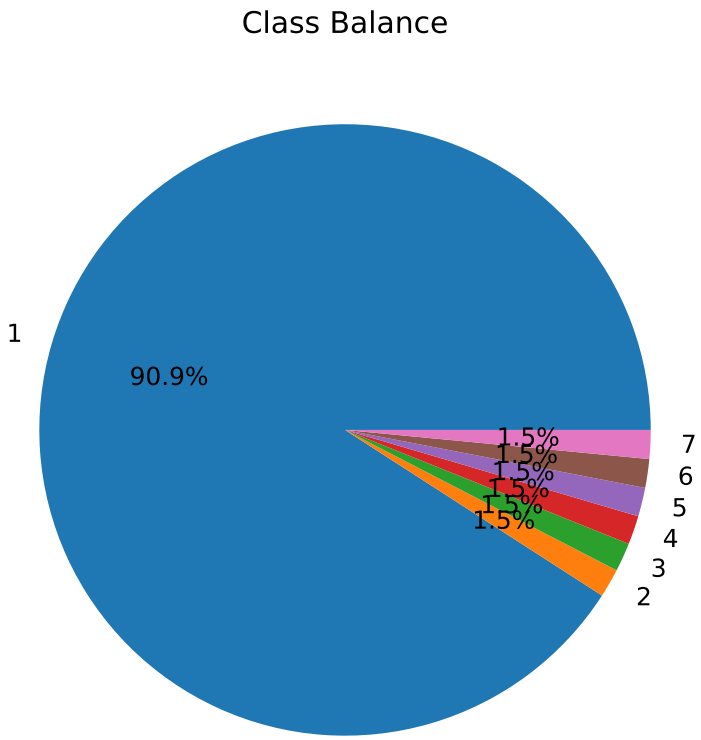

Figure 11:

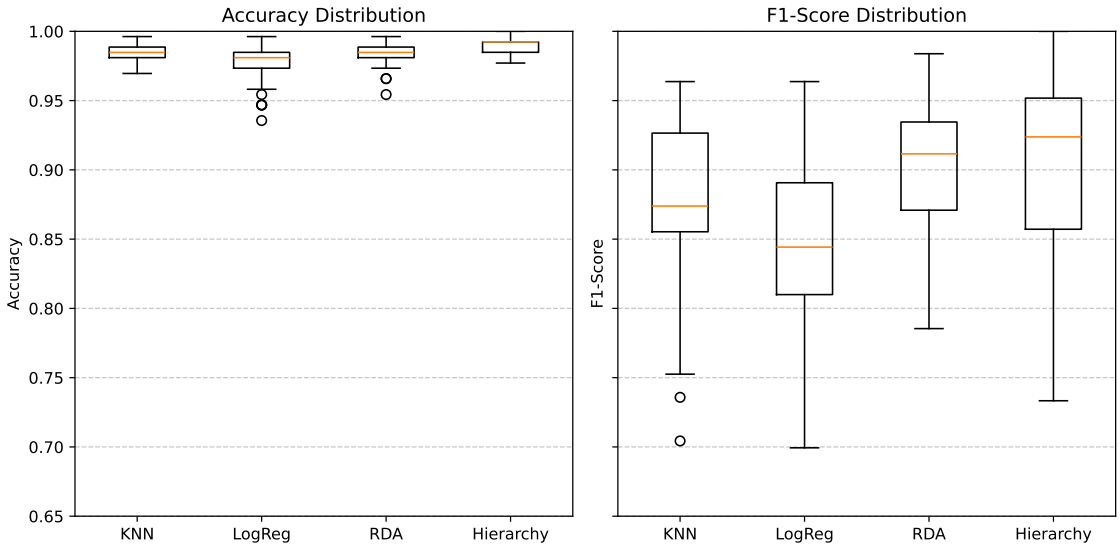

Figure 12:

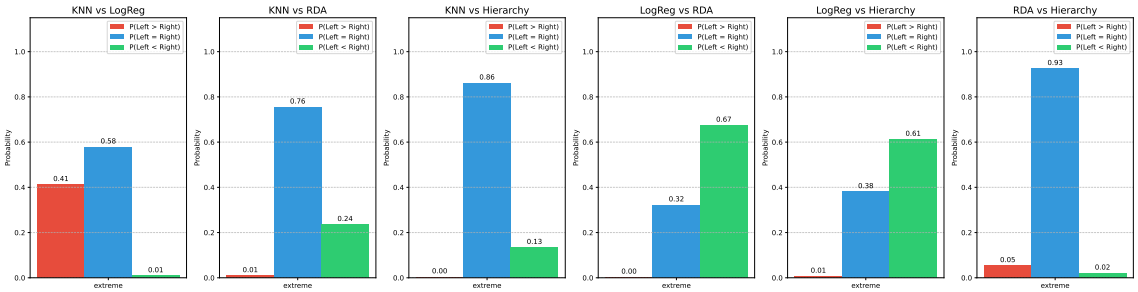
KNN vs LogReg
KNN vs RDA
KNN vs Hierarchy
LogReg vs RDA
LogReg vs Hierarchy
RDA vs Hierarchy
P(Left > Right)
P(Left = Right)
P(Left < Right)
Probability
extreme
0.41
0.58
0.01
0.01
0.76
0.24
0.00
0.86
0.13
0.00
0.32
0.67
0.01
0.38
0.61
0.05
0.93
0.02

Figure 13: